



ESPECIFICACIÓN DE REQUERIMIENTOS DE SOFTWARE (SRS)

Proyecto KalliCode



ALEJANDRO GONZALEZ – ANA PANES – LUIS IBAÑEZ
CAPSTONE

Contenido

1. Introducción	2
1.1. Propósito del Documento	2
1.2. Alcance del Sistema	2
1.3. Definiciones, Acrónimos y Abreviaturas	3
1.4. Visión General del Documento	4
2. Descripción General.....	4
2.1. Perspectiva del Producto	4
2.2. Funciones Principales del Sistema	4
2.3. Características de los Usuarios y Actores.....	5
2.4. Restricciones de Diseño e Infraestructura	5
2.5. Suposiciones y Dependencias.....	5
3. Requerimientos Específicos Funcionales (RFF)	6
4. Requerimientos No Funcionales (RNF)	7

1. Introducción

KalliCode representa un cambio de paradigma en la automatización del ciclo de vida de desarrollo de software (SDLC). Mientras que los asistentes de código tradicionales (Copilots) se centran exclusivamente en acelerar la escritura de líneas, KalliCode ataca el cuello de botella crítico de la industria: la entrega (delivery). La plataforma orquesta una Célula Agéntica que automatiza desde la ingesta de requerimientos hasta el despliegue verificado, resolviendo fricciones en QA, seguridad y cumplimiento normativo que suelen dilatar los lanzamientos por semanas. Este documento de Especificación de Requerimientos de Software (SRS) constituye el marco técnico y estratégico obligatorio para garantizar la integridad arquitectónica del sistema dentro del Portafolio de Título de Ingeniería Informática, sirviendo como la "fuente de verdad" para la validación de sus capacidades disruptivas.

1.1. Propósito del Documento

El presente SRS define de forma formal y exhaustiva los requerimientos funcionales y no funcionales que rigen el ecosistema KalliCode. Su objetivo es servir como base contractual y técnica para el desarrollo, despliegue y validación de la plataforma, asegurando que el diseño de multi-tenencia, la gobernanza criptográfica y la automatización agéntica cumplan con los estándares de ingeniería de software más rigurosos.

1.2. Alcance del Sistema

KalliCode se define como una Célula Agéntica de Entrega de Software SaaS Multi-Tenant. El sistema normaliza payloads de múltiples proveedores (Jira, GitHub, etc.) en un flujo de eventos unificado para su procesamiento idempotente. A través de un pipeline de 6 etapas automatizadas (Triage, Diseño/Spec, Build, QA/Testing, Security, Deploy), la plataforma transforma tickets en Pull Requests (PRs) verificados. El sistema integra una gobernanza humana soberana mediante tres

"Gates" criptográficos vinculantes, asegurando que la autonomía de la IA opere bajo supervisión técnica inmutable.

1.3. Definiciones, Acrónimos y Abreviaturas

- API (Application Programming Interface): Interfaz de programación distribuida en 4 bloques (/api/v1, /api/v1/webhooks, /internal/v1, /admin/v1).
- CodeMapping: Grafo de conocimiento vivo con estructura de 4 capas: Ontología de Negocio, Definiciones Técnicas, Dependencias de Código y Capa Vectorial (pgvector).
- JWT (JSON Web Token): Estándar de autenticación con transición RS256/HS256 y refresh tokens rotativos.
- LLM (Large Language Model): Modelos orquestados en 3 tiers: Flash (tareas rápidas), Pro (razonamiento), Fable (codificación compleja).
- Multi-tenancy: Arquitectura que garantiza el aislamiento lógico de múltiples clientes sobre una infraestructura compartida.
- RLS (Row-Level Security): Mecanismo de PostgreSQL 16 que fuerza el aislamiento a nivel de motor; un recurso ajeno devuelve Error 404 para ocultar su existencia.
- SHA-256: Algoritmo de hash para el sellado inmutable de la cadena de auditoría y firmas de Gates.
- SRS: Software Requirements Specification (este documento).
- ULID (Universally Unique Lexicographically Sortable Identifier): Identificadores únicos con prefijos semánticos obligatorios (u_, org_, att_, job_, st_).
- WORM (Write Once, Read Many): Política de inmutabilidad para almacenamiento de evidencias, equivalente a Object-Lock en entornos MinIO/Azure.

1.4. Visión General del Documento

Este SRS sigue la estructura del estándar IEEE 830. La sección 2 describe la arquitectura de tres capas y las restricciones de infraestructura. La sección 3 detalla los 15 requerimientos funcionales clave del pipeline agéntico. La sección 4 establece los requerimientos no funcionales de alta ingeniería que garantizan la soberanía de datos y la resiliencia del sistema.

2. Descripción General

La arquitectura de KalliCode se fundamenta en un ecosistema de tres capas (API, Workers, Core) diseñado para la orquestación asíncrona y el aislamiento estricto. A diferencia de las herramientas tradicionales, KalliCode separa la ingesta de señales de la ejecución de lógica pesada mediante Redis Streams, permitiendo una escala SaaS donde la infraestructura soporta de forma nativa la soberanía de datos y la inmutabilidad de evidencias.

2.1. Perspectiva del Producto

El sistema opera sobre un stack de alta disponibilidad basado en FastAPI y PostgreSQL 16.

- Estructura de Datos: Segmentada en esquemas core (negocio), vec (búsqueda semántica) y audit (trazabilidad inmutable).
- Orquestación: Redis Streams gestiona la comunicación asíncrona entre el orquestador y los agentes.
- Almacenamiento: Azurite/Azure Blob Storage con políticas WORM para evitar la alteración de evidencias de QA y logs de auditoría. En despliegues soberanos, se utiliza el stack "Sovereign": MinIO, Neo4j y Qdrant.

2.2. Funciones Principales del Sistema

El sistema ejecuta un pipeline de 6 estaciones especializadas: Triage (clasificación), Diseño/Spec (arquitectura), Build (codificación), QA/Testing

(validación), Security (SAST/SCA) y Deploy. La transición entre fases críticas está protegida por 3 Cripto-Gates: Firma de Spec (antes de codificar), Evidencia de QA (antes de Staging) y Firma de Arquitecto (antes de Producción). Cada Gate es una transacción criptográfica grabada en `gate_signatures`.

2.3. Características de los Usuarios y Actores

Actor	Responsabilidad Crítica
Cliente (Portal E4)	Supervisión de tickets y consumo; consulta de expedientes de auditoría.
Agente IA (Fabrica E2)	Ejecución autónoma de tareas agénticas contra el grafo CodeMapping.
Aprobador (Arquitecto E3)	Validación humana vinculante de Gates mediante evidencia visual y técnica.
Admin (kallicode-admin)	Única excepción de RLS, regida por 4 reglas: datos enmascarados, concesiones justificadas con vencimiento, sellado de acciones y visibilidad total para el cliente.

2.4. Restricciones de Diseño e Infraestructura

- Aislamiento de Motor: RLS obligatorio en Postgres 16; las consultas cross-tenant deben fallar con 404 para mitigar ataques de enumeración.
- QA Sandbox: Ejecución de código generado en contenedores efímeros, sin red saliente para prevenir exfiltración de propiedad intelectual.
- Inmutabilidad: Las tablas del esquema audit son append-only; cualquier intento de modificación dispara una alerta CRITICAL.

2.5. Suposiciones y Dependencias

El sistema depende de la disponibilidad de modelos LLM tiers (Flash, Pro, Fable) y su capacidad de escalado automático. Se asume la integración con GitHub/Jira

mediante webhooks firmados con HMAC-SHA256 y el uso de un orquestador determinista para la ejecución de pasos agénticos.

3. Requerimientos Específicos Funcionales (RFF)

KalliCode opera mediante un flujo de datos unidireccional donde la IA actúa como ejecutor de alta velocidad y el humano como validador soberano. El núcleo de la lógica reside en el grafo CodeMapping, que elimina alucinaciones al proveer contexto profundo (200k+ nodos) antes de cualquier generación de código.

- [RFF-01]: Ingesta de tickets y webhooks - Normalización de payloads multi-proveedor en la tabla webhook_inbox. Requiere Idempotency-Key obligatoria para prevenir duplicidad en reintentos de red.
- [RFF-02]: Etapa 01 - Triage Automatizado - Clasificación mediante embeddings BGE-M3 (1024 dims). Genera estimaciones de impacto y detecta duplicados vectoriales comparando contra el historial del tenant.
- [RFF-03]: Etapa 02 - Diseño / Spec - Producción de especificaciones técnicas basadas en las capas de CodeMapping (Ontología, Definiciones, Dependencias y Capa Vectorial).
- [RFF-04]: Gate 1 - Aprobación Humana de Spec - Transacción criptográfica obligatoria. La firma genera un hash SHA-256 en gate_signatures que bloquea el inicio del Build hasta su ejecución.
- [RFF-05]: Etapa 03 - Build - Generación aislada de código fuente, creación de diffs atómicos y apertura de Pull Request mediante agentes especialistas.
- [RFF-06]: Etapa 04 - QA & Testing - Ejecución de planes de prueba (Playwright/Pytest) en contenedores sin red. Captura de trazas y screenshots inmutables almacenados con política WORM.
- [RFF-07]: Gate 2 - Aprobación de QA - Validación humana basada en evidencias visuales reales. El aprobador certifica la matriz de cumplimiento funcional antes de permitir el paso a seguridad.

- [RFF-08]: Etapa 05 - Security - Análisis estático (SAST), de composición (SCA) y detección de secretos en el diff generado. Los hallazgos se consolidan en la pestaña de Seguridad del ticket.
- [RFF-09]: Gate 3 - Firma de Arquitecto - Validación técnica final. Crea un sello inmutable en la cadena de auditoría que vincula el código final con el expediente completo del ticket.
- [RFF-10]: Etapa 06 - Deploy y Rollback - Despliegue automatizado del PR tras Gate 3. Monitoreo de salud inmediato con capacidad de reversión vía `/internal/v1/jobs/{id}/state`.
- [RFF-11]: Autenticación y Sesión - Uso de Argon2id para hashes de contraseñas. JWT con RS256 y refresh tokens rotativos persistidos en base de datos para revocación inmediata.
- [RFF-12]: Gobernanza de Cuotas Comerciales - Control de límites (Starter: 3,000 LOC; Growth: 8,000 LOC netas/mes). Una Línea de Producción operativa equivale al output mensual de un Desarrollador Senior.
- [RFF-13]: Superficies de API FastAPI - Exposición de 115 endpoints distribuidos en: `/api/v1` (Portal), `/api/v1/webhooks` (Integraciones), `/internal/v1` (Pipeline) y `/admin/v1` (Consola).
- [RFF-14]: Gestión de Expedientes de Auditoría - Trazabilidad total de cada paso LLM y firma humana en `audit.audit_events`, protegida por una cadena de hashes encadenados.
- [RFF-15]: Consola Administrativa Cross-Tenant - Herramienta para `kallicode-admin`. Implementa enmascarado por defecto y exige motivo escrito para revelar datos de clientes, con visibilidad de estas acciones para el cliente final.

4. Requerimientos No Funcionales (RNF)

Los requerimientos no funcionales en KalliCode constituyen las restricciones duras del sistema, garantizando que la multi-tenencia y la seguridad se ejecuten a nivel de motor (Engine) y no dependan de la capa de aplicación.

- [RNF-01]: Aislamiento Multi-Tenant Estricto - Implementación de RLS en PostgreSQL 16 utilizando los roles kallicode_app y kallicode_system_role. El sistema debe superar la puerta de integración "no-fuga", devolviendo 404 ante cualquier intento de acceso cross-tenant.
- [RNF-02]: Criptografía e Inmutabilidad - Cadena de auditoría append-only en audit.audit_events sellada por triggers. La rotura de un solo eslabón de la cadena de hashes SHA-256 debe disparar una alerta CRITICAL y bloquear operaciones de seguridad.
- [RNF-03]: Seguridad en Sandbox de QA - Aislamiento total de red (Air-gapped) en contenedores de ejecución. Los contenedores son efímeros y se destruyen tras cada corrida de pruebas para evitar persistencia de estados.
- [RNF-04]: Rendimiento y Latencia - Tiempo de respuesta de API < 200ms (p95). El procesamiento pesado se delega a Workers mediante Redis Streams para mantener la reactividad del Portal.
- [RNF-05]: Disponibilidad y Resiliencia - SLA del 99.9%. Implementación obligatoria de reintentos exponenciales y Circuit Breakers para todas las llamadas a proveedores de LLM.
- [RNF-06]: Mantenibilidad y Calidad - Cobertura de tests $\geq 80\%$. Linting estricto con Ruff, tipado estático con Mypy(strict) y cumplimiento de convenciones de arquitectura limpia.
- [RNF-07]: Escalabilidad de Almacenamiento - Búsquedas vectoriales optimizadas mediante indexación HNSW en pgvector. Gestión de ciclo de vida de evidencias en Blob Storage bajo políticas de retención legal.
- [RNF-08]: Soberanía de Datos - Soporte de "Sovereign Stack" para despliegues On-Premise: PostgreSQL soberano, MinIO para objetos, Neo4j para el grafo, Qdrant para vectores y LLMs locales para cero exfiltraciones.
- [RNF-09]: Trazabilidad y Observabilidad - Logging estructurado JSON con trace_id y tenant_id obligatorio. Las trazas deben permitir reconstruir el flujo completo de un ticket a través de las 6 etapas.

- [RNF-10]: Tolerancia a Fallos - Ante errores agénticos no recuperables, el sistema debe realizar una escalación automática al endpoint /internal/v1/escalations, notificando al humano responsable para intervención manual.

Este documento SRS constituye el compromiso técnico final para la entrega de la Célula Agéntica KalliCode, integrando velocidad de máquina con la más alta gobernanza humana y rigor de ingeniería.